

K13A-GRID Analysis: GHK vs GQT Across the Parameter Grid

STAT778 Project

2026-04-24

Contents

What this Rmd does	1
Section 1: Load and tidy	1
Load every cell	1
Extract cell parameters from each scenario_id	2
Tidy to long format	2
Attach truth	4
Section 2: Convergence diagnostics	4
Section 3: Runtime comparison	5
Headline: GHK vs GQT speed ratio	5
Plot: runtime by cell	6
Section 4: Parameter estimates	6
Means and SDs per cell	6
Bias (signed error)	7
Section 5: MAE — the headline table and figure	9
MAE per cell	9
Direct GHK vs GQT MAE comparison	11
The Figure-1 equivalent: MAE vs sample size	12
Save the figures	14
Section 6: Summary for the report	14
Likely findings to report	15
Section 7: Tables for the report	15
End of K13A-GRID analysis	17

What this Rmd does

Analyzes the 18-cell grid study (Day 4): 2 sample sizes x 3 ranges x 3 lambda values, each with 200 replicates fit by both GHK and GQT. Produces the tables and figures that anchor the report's simulation results section.

Organization. Section 1 tidies the raw fit results into a long data frame. Sections 2-5 produce specific outputs (convergence, runtime, estimates, MAE). Section 6 summarizes findings.

Section 1: Load and tidy

Load every cell

```
source("code/R/logging.R")
source("code/R/io.R")
source("code/R/fit_core.R")
```

```
DATASET <- "K13A-GRID"

ids <- list_scenarios(DATASET, dir = "data/generated")
length(ids)  # should be 18

## [1] 18

# Load all fit results. Each one is a list with dataset_id,
# scenario_id, results (data frame), and metadata.
fits <- lapply(ids, function(id) load_fit_result(DATASET, id))
names(fits) <- ids
```

Quick sanity check — same shape everywhere?

```
sapply(fits, function(f) nrow(f$results))  # should all be 400 (200 x 2)

## pois_n25_r0.1_g1 pois_n25_r0.1_g10 pois_n25_r0.1_g5 pois_n25_r0.3_g1
##               400               400               400               400
## pois_n25_r0.3_g10 pois_n25_r0.3_g5 pois_n25_r0.5_g1 pois_n25_r0.5_g10
##               400               400               400               400
## pois_n25_r0.5_g5 pois_n49_r0.1_g1 pois_n49_r0.1_g10 pois_n49_r0.1_g5
##               400               400               400               400
## pois_n49_r0.3_g1 pois_n49_r0.3_g10 pois_n49_r0.3_g5 pois_n49_r0.5_g1
##               400               400               400               400
## pois_n49_r0.5_g10 pois_n49_r0.5_g5
##               400               400
```

Extract cell parameters from each scenario_id

The scenario_id encodes (n, range, lambda) so we don't need to dig into each config. But parsing out of the filename is a fragile convention — better to pull from the stored config.

```
cell_info <- do.call(rbind, lapply(fits, function(f) {
  cfg <- f$scenario_config
  data.frame(
    scenario_id = cfg$scenario_id,
    n_total     = cfg$layout$n_side^2,
    range       = cfg$corr$range,
    lambda      = cfg$marginal$lambda,
    stringsAsFactors = FALSE
  )
}))

head(cell_info)
```

	scenario_id	n_total	range	lambda
## pois_n25_r0.1_g1	pois_n25_r0.1_g1	25	0.1	1
## pois_n25_r0.1_g10	pois_n25_r0.1_g10	25	0.1	10
## pois_n25_r0.1_g5	pois_n25_r0.1_g5	25	0.1	5
## pois_n25_r0.3_g1	pois_n25_r0.3_g1	25	0.3	1
## pois_n25_r0.3_g10	pois_n25_r0.3_g10	25	0.3	10
## pois_n25_r0.3_g5	pois_n25_r0.3_g5	25	0.3	5

Tidy to long format

One row per (scenario, replicate, method, parameter). This is the shape that makes aggregation easy.

```

tidy_one <- function(f) {
  r <- f$results
  cfg <- f$scenario_config

  rows <- list()
  for (i in seq_len(nrow(r))) {
    if (!r$converged[i]) {
      # record a single "failed" row so failures are visible
      rows[[length(rows) + 1]] <- data.frame(
        scenario_id = f$scenario_id,
        n_total     = cfg$layout$n_side^2,
        range       = cfg$corr$range,
        lambda      = cfg$marginal$lambda,
        replicate    = r$replicate[i],
        method       = r$method[i],
        converged     = FALSE,
        runtime_sec  = r$runtime_sec[i],
        parameter    = NA_character_,
        estimate      = NA_real_,
        stringsAsFactors = FALSE
      )
      next
    }

    est <- r$estimates[[i]]
    if (is.null(est) || length(est) == 0) next

    for (pname in names(est)) {
      rows[[length(rows) + 1]] <- data.frame(
        scenario_id = f$scenario_id,
        n_total     = cfg$layout$n_side^2,
        range       = cfg$corr$range,
        lambda      = cfg$marginal$lambda,
        replicate    = r$replicate[i],
        method       = r$method[i],
        converged     = TRUE,
        runtime_sec  = r$runtime_sec[i],
        parameter    = pname,
        estimate      = as.numeric(est[[pname]]),
        stringsAsFactors = FALSE
      )
    }
  }

  do.call(rbind, rows)
}

long <- do.call(rbind, lapply(fits, tidy_one))
nrow(long)

```

```
## [1] 14398
```

```
head(long)
```

```
##                scenario_id n_total range lambda replicate method
## pois_n25_r0.1_g1.1 pois_n25_r0.1_g1      25  0.1      1        1   GHK
## pois_n25_r0.1_g1.2 pois_n25_r0.1_g1      25  0.1      1        1   GHK
## pois_n25_r0.1_g1.3 pois_n25_r0.1_g1      25  0.1      1        1   GQT
## pois_n25_r0.1_g1.4 pois_n25_r0.1_g1      25  0.1      1        1   GQT
## pois_n25_r0.1_g1.5 pois_n25_r0.1_g1      25  0.1      1        2   GHK
## pois_n25_r0.1_g1.6 pois_n25_r0.1_g1      25  0.1      1        2   GHK
##                converged runtime_sec parameter      estimate
## pois_n25_r0.1_g1.1      TRUE  0.56232715 Intercept -0.44628716
## pois_n25_r0.1_g1.2      TRUE  0.56232715      range  0.00000000
## pois_n25_r0.1_g1.3      TRUE  0.03048372 Intercept -0.44628727
## pois_n25_r0.1_g1.4      TRUE  0.03048372      range  0.00000000
## pois_n25_r0.1_g1.5      TRUE  0.16973805 Intercept  0.03922065
## pois_n25_r0.1_g1.6      TRUE  0.16973805      range  0.00000000
```

Attach truth

```
# gcKrig's Poisson intercept is on the log scale; truth is log(lambda).
long$truth <- NA_real_
long$truth[long$parameter %in% "Intercept"] <- log(long$lambda[long$parameter %in% "Intercept"])
long$truth[long$parameter %in% "range"]      <- long$range[long$parameter %in% "range"]

long$abs_err <- abs(long$estimate - long$truth)
long$err      <- long$estimate - long$truth # signed, for bias

head(long[, c("scenario_id", "method", "parameter", "estimate", "truth", "err")])

##                scenario_id method parameter      estimate truth
## pois_n25_r0.1_g1.1 pois_n25_r0.1_g1   GHK Intercept -0.44628716  0.0
## pois_n25_r0.1_g1.2 pois_n25_r0.1_g1   GHK      range  0.00000000  0.1
## pois_n25_r0.1_g1.3 pois_n25_r0.1_g1   GQT Intercept -0.44628727  0.0
## pois_n25_r0.1_g1.4 pois_n25_r0.1_g1   GQT      range  0.00000000  0.1
## pois_n25_r0.1_g1.5 pois_n25_r0.1_g1   GHK Intercept  0.03922065  0.0
## pois_n25_r0.1_g1.6 pois_n25_r0.1_g1   GHK      range  0.00000000  0.1
##                err
## pois_n25_r0.1_g1.1 -0.44628716
## pois_n25_r0.1_g1.2 -0.10000000
## pois_n25_r0.1_g1.3 -0.44628727
## pois_n25_r0.1_g1.4 -0.10000000
## pois_n25_r0.1_g1.5  0.03922065
## pois_n25_r0.1_g1.6 -0.10000000
```

Section 2: Convergence diagnostics

```
conv_tab <- aggregate(
  converged ~ n_total + range + lambda + method,
  data = unique(long[, c("scenario_id", "replicate", "method",
                        "n_total", "range", "lambda", "converged")]),
  FUN = function(x) sum(x) / length(x)
)
```

```
names(conv_tab)[5] <- "conv_rate"

# Flag any cell below 99% convergence
conv_tab[conv_tab$conv_rate < 0.99, ]

## [1] n_total range lambda method conv_rate
## <0 rows> (or 0-length row.names)

If that's empty, every cell had at least 99% convergence. Good.
cat("Overall convergence by method:\n")

## Overall convergence by method:
print(aggregate(conv_rate ~ method, data = conv_tab, FUN = mean))

## method conv_rate
## 1 GHK 1.0000000
## 2 GQT 0.9994444
```

Section 3: Runtime comparison

```
rt <- long[long$converged, ]
rt <- unique(rt[, c("scenario_id", "replicate", "method", "n_total",
                    "range", "lambda", "runtime_sec")])

runtime_summary <- aggregate(
  runtime_sec ~ n_total + method,
  data = rt,
  FUN = function(x) c(mean = mean(x), median = median(x),
                      min = min(x), max = max(x))
)
runtime_summary

## n_total method runtime_sec.mean runtime_sec.median runtime_sec.min
## 1 25 GHK 0.27008934 0.24803805 0.14746118
## 2 49 GHK 0.69836656 0.63883114 0.35645199
## 3 25 GQT 0.03155089 0.03090823 0.01409340
## 4 49 GQT 0.08464145 0.08431756 0.03313017
## runtime_sec.max
## 1 1.12196231
## 2 2.71230507
## 3 0.11222529
## 4 0.26077223
```

Headline: GHK vs GQT speed ratio

```
rt_means <- aggregate(runtime_sec ~ n_total + method, data = rt, FUN = mean)
rt_wide <- reshape(rt_means, idvar = "n_total", timevar = "method",
                  direction = "wide")
rt_wide$ratio_GHK_to_GQT <- rt_wide$runtime_sec.GHK / rt_wide$runtime_sec.GQT
rt_wide

## n_total runtime_sec.GHK runtime_sec.GQT ratio_GHK_to_GQT
```

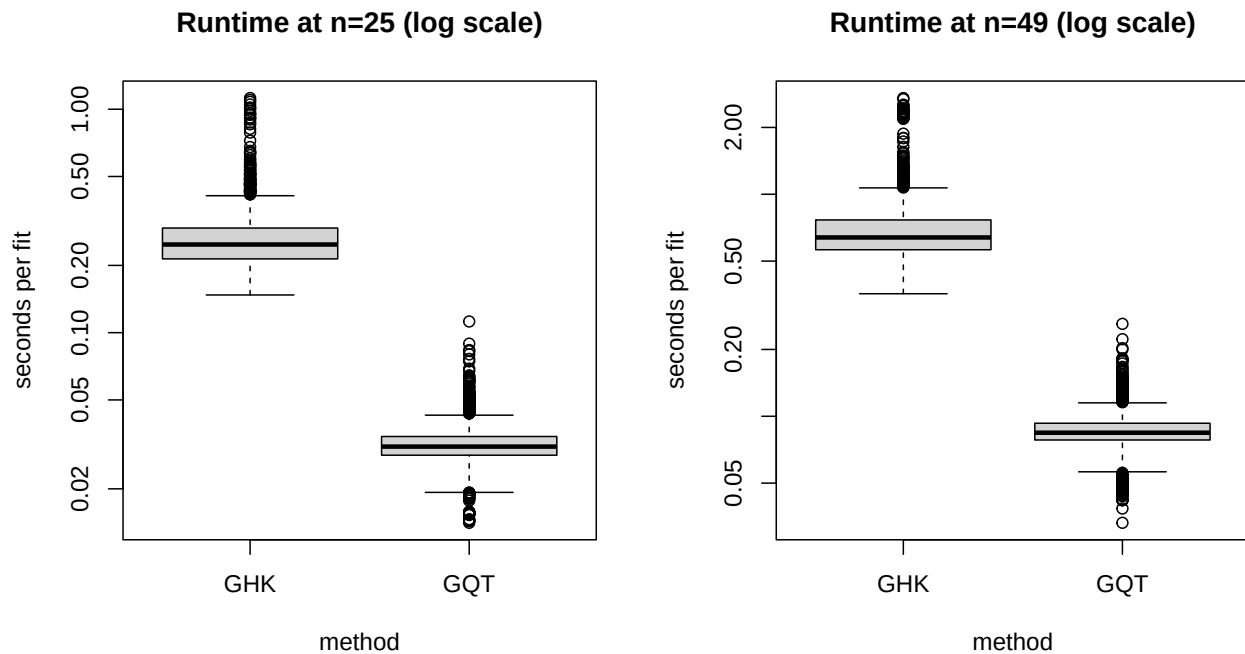
## 1	25	0.2700893	0.03155089	8.560436
## 2	49	0.6983666	0.08464145	8.250881

This is the number for the report: “GQT is roughly Xx faster than GHK at n=25, and Yx faster at n=49.”

Plot: runtime by cell

```
par(mfrow = c(1, 2))

for (n_val in sort(unique(rt$n_total))) {
  sub <- rt[rt$n_total == n_val, ]
  boxplot(
    runtime_sec ~ method, data = sub,
    log = "y",
    main = sprintf("Runtime at n=%d (log scale)", n_val),
    ylab = "seconds per fit"
  )
}
```



```
par(mfrow = c(1, 1))
```

Section 4: Parameter estimates

Means and SDs per cell

```
est_summary <- aggregate(
  estimate ~ n_total + range + lambda + method + parameter,
  data = long[long$converged, ],
  FUN = function(x) c(mean = mean(x), sd = sd(x), n = length(x))
)

# flatten the aggregated value column for display
```

```
est_display <- do.call(data.frame, est_summary)
head(est_display)
```

```
##   n_total range lambda method parameter estimate.mean estimate.sd estimate.n
## 1      25   0.1      1   GHK Intercept  -0.01569976   0.2423074         200
## 2      49   0.1      1   GHK Intercept  -0.02556882   0.2143656         200
## 3      25   0.3      1   GHK Intercept  -0.09982922   0.3997500         200
## 4      49   0.3      1   GHK Intercept  -0.10496116   0.3850063         200
## 5      25   0.5      1   GHK Intercept  -0.22133655   1.8559003         200
## 6      49   0.5      1   GHK Intercept  -0.09083221   0.5509213         200
```

Bias (signed error)

```
bias_tab <- aggregate(
  err ~ n_total + range + lambda + method + parameter,
  data = long[long$converged, ],
  FUN = mean
)
names(bias_tab)[6] <- "bias"

# Split by parameter for readability
bias_intercept <- bias_tab[bias_tab$parameter == "Intercept", ]
bias_range <- bias_tab[bias_tab$parameter == "range", ]

cat("Bias for log-Intercept:\n")
```

```
## Bias for log-Intercept:
```

```
print(bias_intercept, row.names = FALSE)
```

```
##   n_total range lambda method parameter      bias
##      25   0.1      1   GHK Intercept -0.0156997616
##      49   0.1      1   GHK Intercept -0.0255688233
##      25   0.3      1   GHK Intercept -0.0998292157
##      49   0.3      1   GHK Intercept -0.1049611566
##      25   0.5      1   GHK Intercept -0.2213365492
##      49   0.5      1   GHK Intercept -0.0908322061
##      25   0.1      5   GHK Intercept -0.0135243368
##      49   0.1      5   GHK Intercept -0.0083433778
##      25   0.3      5   GHK Intercept -0.0472502113
##      49   0.3      5   GHK Intercept -0.0359307081
##      25   0.5      5   GHK Intercept -0.0614071094
##      49   0.5      5   GHK Intercept -0.0527259873
##      25   0.1     10   GHK Intercept  0.0007880722
##      49   0.1     10   GHK Intercept  0.0014596740
##      25   0.3     10   GHK Intercept -0.0143916970
##      49   0.3     10   GHK Intercept -0.0227353586
##      25   0.5     10   GHK Intercept -0.0195224041
##      49   0.5     10   GHK Intercept  0.0009896270
##      25   0.1      1   GQT Intercept -0.0164409708
##      49   0.1      1   GQT Intercept -0.0242187716
##      25   0.3      1   GQT Intercept -0.0536405320
##      49   0.3      1   GQT Intercept -0.0332967067
##      25   0.5      1   GQT Intercept  0.0021484820
```

```
##      49    0.5      1    GQT Intercept  0.0539092523
##      25    0.1      5    GQT Intercept -0.0132177496
##      49    0.1      5    GQT Intercept -0.0085766304
##      25    0.3      5    GQT Intercept -0.0466653854
##      49    0.3      5    GQT Intercept -0.0368970961
##      25    0.5      5    GQT Intercept -0.0599172122
##      49    0.5      5    GQT Intercept -0.0430555840
##      25    0.1     10    GQT Intercept  0.0006111759
##      49    0.1     10    GQT Intercept  0.0009991307
##      25    0.3     10    GQT Intercept -0.0135141511
##      49    0.3     10    GQT Intercept -0.0224906889
##      25    0.5     10    GQT Intercept -0.0179205067
##      49    0.5     10    GQT Intercept  0.0038641825
```

```
cat("\nBias for range:\n")
```

```
##
```

```
## Bias for range:
```

```
print(bias_range, row.names = FALSE)
```

```
##  n_total range lambda method parameter      bias
##      25    0.1      1    GHK      range -0.0338784980
##      49    0.1      1    GHK      range -0.0378033202
##      25    0.3      1    GHK      range -0.0337454048
##      49    0.3      1    GHK      range -0.0183187074
##      25    0.5      1    GHK      range -0.0285692130
##      49    0.5      1    GHK      range -0.0248302863
##      25    0.1      5    GHK      range -0.0359856820
##      49    0.1      5    GHK      range -0.0493820510
##      25    0.3      5    GHK      range  0.0080297040
##      49    0.3      5    GHK      range -0.0124503448
##      25    0.5      5    GHK      range -0.0015123815
##      49    0.5      5    GHK      range  0.0052479860
##      25    0.1     10    GHK      range -0.0236751392
##      49    0.1     10    GHK      range -0.0495401993
##      25    0.3     10    GHK      range -0.0224589315
##      49    0.3     10    GHK      range -0.0018161555
##      25    0.5     10    GHK      range  0.0180105600
##      49    0.5     10    GHK      range -0.0011619707
##      25    0.1      1    GQT      range  0.0170093818
##      49    0.1      1    GQT      range  0.0193606425
##      25    0.3      1    GQT      range  0.0579009226
##      49    0.3      1    GQT      range  0.0472290323
##      25    0.5      1    GQT      range  0.0853612281
##      49    0.5      1    GQT      range  0.0530988133
##      25    0.1      5    GQT      range -0.0313403149
##      49    0.1      5    GQT      range -0.0529578933
##      25    0.3      5    GQT      range  0.0179716489
##      49    0.3      5    GQT      range -0.0082294760
##      25    0.5      5    GQT      range  0.0058036958
##      49    0.5      5    GQT      range  0.0040668939
##      25    0.1     10    GQT      range -0.0195802685
##      49    0.1     10    GQT      range -0.0489726923
##      25    0.3     10    GQT      range -0.0181070526
```



```
##      49    0.3    10    GQT    range -0.0001560223
##      25    0.5    10    GQT    range  0.0199914145
##      49    0.5    10    GQT    range -0.0021083054
```

What to look for. Values close to zero mean the estimate is on average correct. Range estimates tend to be *negatively biased* — finite samples underestimate spatial dependence. Intercept estimates are usually closer to unbiased.

Section 5: MAE — the headline table and figure

MAE per cell

```
mae_tab <- aggregate(
  abs_err ~ n_total + range + lambda + method + parameter,
  data = long[long$converged, ],
  FUN = mean
)
names(mae_tab)[6] <- "MAE"

# Two sub-tables for the report: one per parameter.
mae_intercept <- mae_tab[mae_tab$parameter == "Intercept", ]
mae_range      <- mae_tab[mae_tab$parameter == "range", ]

cat("MAE for log-Intercept:\n")

## MAE for log-Intercept:
print(mae_intercept[order(mae_intercept$n_total, mae_intercept$range,
  mae_intercept$lambda, mae_intercept$method), ],
  row.names = FALSE, digits = 3)
```

##	n_total	range	lambda	method	parameter	MAE
##	25	0.1	1	GHK	Intercept	0.1943
##	25	0.1	1	GQT	Intercept	0.1896
##	25	0.1	5	GHK	Intercept	0.0860
##	25	0.1	5	GQT	Intercept	0.0854
##	25	0.1	10	GHK	Intercept	0.0582
##	25	0.1	10	GQT	Intercept	0.0579
##	25	0.3	1	GHK	Intercept	0.3207
##	25	0.3	1	GQT	Intercept	0.2889
##	25	0.3	5	GHK	Intercept	0.1631
##	25	0.3	5	GQT	Intercept	0.1645
##	25	0.3	10	GHK	Intercept	0.1132
##	25	0.3	10	GQT	Intercept	0.1133
##	25	0.5	1	GHK	Intercept	0.5196
##	25	0.5	1	GQT	Intercept	0.3458
##	25	0.5	5	GHK	Intercept	0.2039
##	25	0.5	5	GQT	Intercept	0.2053
##	25	0.5	10	GHK	Intercept	0.1345
##	25	0.5	10	GQT	Intercept	0.1337
##	49	0.1	1	GHK	Intercept	0.1732
##	49	0.1	1	GQT	Intercept	0.1671
##	49	0.1	5	GHK	Intercept	0.0724
##	49	0.1	5	GQT	Intercept	0.0730

```
##      49    0.1     10    GHK Intercept 0.0573
##      49    0.1     10    GQT Intercept 0.0574
##      49    0.3      1    GHK Intercept 0.3103
##      49    0.3      1    GQT Intercept 0.2693
##      49    0.3      5    GHK Intercept 0.1537
##      49    0.3      5    GQT Intercept 0.1550
##      49    0.3     10    GHK Intercept 0.1016
##      49    0.3     10    GQT Intercept 0.1027
##      49    0.5      1    GHK Intercept 0.4129
##      49    0.5      1    GQT Intercept 0.3550
##      49    0.5      5    GHK Intercept 0.2034
##      49    0.5      5    GQT Intercept 0.2006
##      49    0.5     10    GHK Intercept 0.1396
##      49    0.5     10    GQT Intercept 0.1399
```

```
cat("\nMAE for range:\n")
```

```
##
```

```
## MAE for range:
```

```
print(mae_range[order(mae_range$n_total, mae_range$range,
                      mae_range$lambda, mae_range$method), ],
      row.names = FALSE, digits = 3)
```

```
##  n_total range lambda method parameter    MAE
##      25    0.1      1    GHK      range 0.0972
##      25    0.1      1    GQT      range 0.1085
##      25    0.1      5    GHK      range 0.1010
##      25    0.1      5    GQT      range 0.1000
##      25    0.1     10    GHK      range 0.0951
##      25    0.1     10    GQT      range 0.0948
##      25    0.3      1    GHK      range 0.1632
##      25    0.3      1    GQT      range 0.1596
##      25    0.3      5    GHK      range 0.1192
##      25    0.3      5    GQT      range 0.1155
##      25    0.3     10    GHK      range 0.1314
##      25    0.3     10    GQT      range 0.1299
##      25    0.5      1    GHK      range 0.2207
##      25    0.5      1    GQT      range 0.2077
##      25    0.5      5    GHK      range 0.1581
##      25    0.5      5    GQT      range 0.1535
##      25    0.5     10    GHK      range 0.1581
##      25    0.5     10    GQT      range 0.1565
##      49    0.1      1    GHK      range 0.0792
##      49    0.1      1    GQT      range 0.0639
##      49    0.1      5    GHK      range 0.0775
##      49    0.1      5    GQT      range 0.0832
##      49    0.1     10    GHK      range 0.0773
##      49    0.1     10    GQT      range 0.0786
##      49    0.3      1    GHK      range 0.0997
##      49    0.3      1    GQT      range 0.0970
##      49    0.3      5    GHK      range 0.0699
##      49    0.3      5    GQT      range 0.0669
##      49    0.3     10    GHK      range 0.0605
##      49    0.3     10    GQT      range 0.0599
```

```
##      49    0.5      1    GHK      range 0.1555
##      49    0.5      1    GQT      range 0.1412
##      49    0.5      5    GHK      range 0.1064
##      49    0.5      5    GQT      range 0.1019
##      49    0.5     10    GHK      range 0.1092
##      49    0.5     10    GQT      range 0.1075
```

Direct GHK vs GQT MAE comparison

The cleanest way to see whether GQT keeps up with GHK everywhere is a wide table: one row per cell, separate columns for each method, and a difference.

```
mae_wide <- reshape(
  mae_tab,
  idvar      = c("n_total", "range", "lambda", "parameter"),
  timevar    = "method",
  direction  = "wide"
)
mae_wide$diff <- mae_wide$MAE.GQT - mae_wide$MAE.GHK
mae_wide$pct_diff <- 100 * mae_wide$diff / mae_wide$MAE.GHK

mae_wide <- mae_wide[order(mae_wide$parameter, mae_wide$n_total,
                          mae_wide$range, mae_wide$lambda), ]
print(mae_wide, row.names = FALSE, digits = 3)
```

```
##  n_total range lambda parameter MAE.GHK MAE.GQT      diff pct_diff
##      25   0.1      1 Intercept  0.1943  0.1896 -4.66e-03  -2.4000
##      25   0.1      5 Intercept  0.0860  0.0854 -5.48e-04  -0.6377
##      25   0.1     10 Intercept  0.0582  0.0579 -2.48e-04  -0.4262
##      25   0.3      1 Intercept  0.3207  0.2889 -3.18e-02  -9.9204
##      25   0.3      5 Intercept  0.1631  0.1645  1.35e-03   0.8285
##      25   0.3     10 Intercept  0.1132  0.1133  1.11e-04   0.0980
##      25   0.5      1 Intercept  0.5196  0.3458 -1.74e-01 -33.4443
##      25   0.5      5 Intercept  0.2039  0.2053  1.42e-03   0.6961
##      25   0.5     10 Intercept  0.1345  0.1337 -8.01e-04  -0.5955
##      49   0.1      1 Intercept  0.1732  0.1671 -6.17e-03  -3.5609
##      49   0.1      5 Intercept  0.0724  0.0730  5.79e-04   0.7992
##      49   0.1     10 Intercept  0.0573  0.0574  3.86e-05   0.0673
##      49   0.3      1 Intercept  0.3103  0.2693 -4.10e-02 -13.2085
##      49   0.3      5 Intercept  0.1537  0.1550  1.26e-03   0.8197
##      49   0.3     10 Intercept  0.1016  0.1027  1.13e-03   1.1078
##      49   0.5      1 Intercept  0.4129  0.3550 -5.79e-02 -14.0165
##      49   0.5      5 Intercept  0.2034  0.2006 -2.78e-03  -1.3644
##      49   0.5     10 Intercept  0.1396  0.1399  2.93e-04   0.2101
##      25   0.1      1      range  0.0972  0.1085  1.13e-02  11.6411
##      25   0.1      5      range  0.1010  0.1000 -1.01e-03  -0.9990
##      25   0.1     10      range  0.0951  0.0948 -3.65e-04  -0.3834
##      25   0.3      1      range  0.1632  0.1596 -3.62e-03  -2.2192
##      25   0.3      5      range  0.1192  0.1155 -3.71e-03  -3.1119
##      25   0.3     10      range  0.1314  0.1299 -1.52e-03  -1.1529
##      25   0.5      1      range  0.2207  0.2077 -1.30e-02  -5.9024
##      25   0.5      5      range  0.1581  0.1535 -4.61e-03  -2.9177
##      25   0.5     10      range  0.1581  0.1565 -1.52e-03  -0.9587
##      49   0.1      1      range  0.0792  0.0639 -1.53e-02 -19.3297
##      49   0.1      5      range  0.0775  0.0832  5.71e-03   7.3712
```

##	49	0.1	10	range	0.0773	0.0786	1.29e-03	1.6657
##	49	0.3	1	range	0.0997	0.0970	-2.72e-03	-2.7270
##	49	0.3	5	range	0.0699	0.0669	-3.00e-03	-4.2944
##	49	0.3	10	range	0.0605	0.0599	-6.73e-04	-1.1113
##	49	0.5	1	range	0.1555	0.1412	-1.43e-02	-9.2175
##	49	0.5	5	range	0.1064	0.1019	-4.47e-03	-4.1968
##	49	0.5	10	range	0.1092	0.1075	-1.64e-03	-1.4984

Report claim would be something like: “Across the 18 cells, GQT’s MAE differs from GHK’s by less than X% for the intercept and less than Y% for the range parameter.”

The Figure-1 equivalent: MAE vs sample size

Kazianka’s paper plots MAE as a function of n , faceted by range, with colored lines for different methods or lambda values. Even with only n in $\{25, 49\}$, the plot is informative.

*# Separate panels for the two parameters. Base-R approach because
the report may ultimately use ggplot for final; this is the
exploratory version.*

```
plot_mae_panel <- function(param_name) {
  dat <- mae_tab[mae_tab$parameter == param_name, ]
  ranges <- sort(unique(dat$range))

  par(mfrow = c(1, length(ranges)),
      oma = c(0, 0, 3, 0), mar = c(4, 4, 3, 1))

  for (r in ranges) {
    d <- dat[dat$range == r, ]
    y_max <- max(d$MAE, na.rm = TRUE) * 1.1

    # One line per method; x = n_total; color by lambda
    plot(NA, xlim = range(d$n_total), ylim = c(0, y_max),
        xlab = "n", ylab = "MAE",
        main = sprintf("range = %g", r))

    lams <- sort(unique(d$lambda))
    cols <- c("firebrick", "steelblue", "darkgreen")
    ltys <- c(1, 2) # solid = GHK, dashed = GQT

    for (li in seq_along(lams)) {
      for (mi in seq_along(c("GHK", "GQT"))) {
        m <- c("GHK", "GQT")[mi]
        sub <- d[d$lambda == lams[li] & d$method == m, ]
        sub <- sub[order(sub$n_total), ]
        lines(sub$n_total, sub$MAE,
            col = cols[li], lty = ltys[mi], lwd = 2)
        points(sub$n_total, sub$MAE,
            col = cols[li], pch = 19)
      }
    }
  }

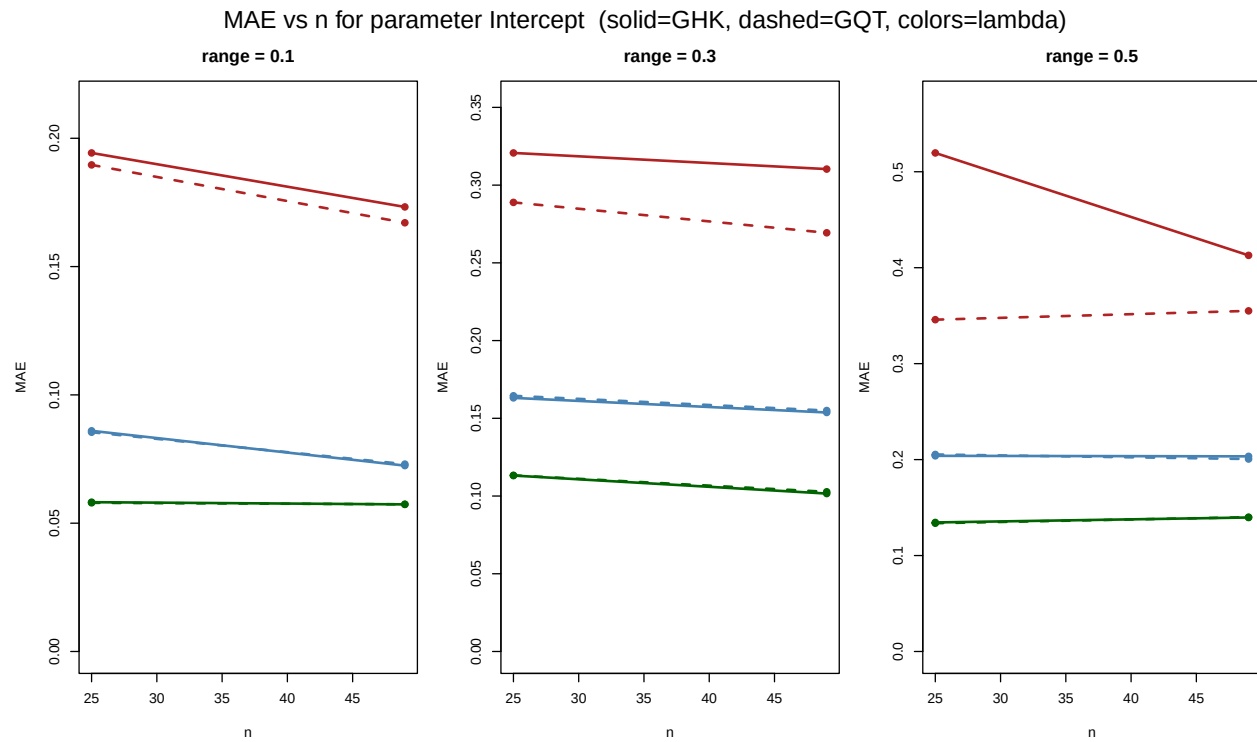
  mtext(sprintf("MAE vs n for parameter %s (solid=GHK, dashed=GQT, colors=lambda)",
      param_name),
```

```

    outer = TRUE, cex = 1.2)
  par(mfrow = c(1, 1))
}

plot_mae_panel("Intercept")

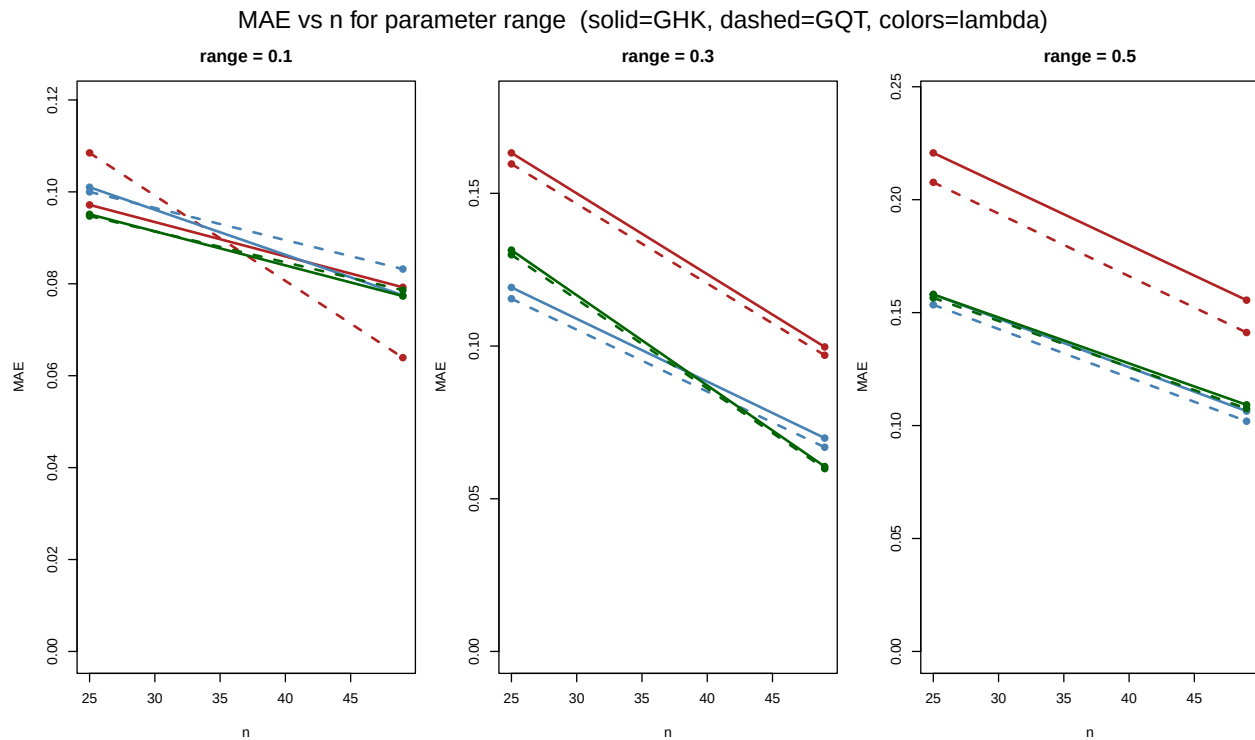
```



```

plot_mae_panel("range")

```



Save the figures

```
dir.create("visualization/k13a_grid", recursive = TRUE, showWarnings = FALSE)

pdf("visualization/k13a_grid/mae_intercept.pdf", width = 10, height = 6)
plot_mae_panel("Intercept")
dev.off()

pdf("visualization/k13a_grid/mae_range.pdf", width = 10, height = 6)
plot_mae_panel("range")
dev.off()

message("Figures saved to visualization/k13a_grid/")
```

Section 6: Summary for the report

Brief checklist of what the results say. Fill in the actual numbers after running the chunks above.

```
# Convergence
cat("Overall convergence:\n")

## Overall convergence:
print(round(tapply(long$converged[long$parameter == "Intercept" | is.na(long$parameter)],
                  long$method[long$parameter == "Intercept" | is.na(long$parameter)],
                  mean), 4))

##      GHK      GQT
## 1.0000 0.9994
```

```

# Runtime speedup
cat("\nMean runtime:\n")

##
## Mean runtime:
print(tapply(rt$runtime_sec, rt$method, mean))

##          GHK          GQT
## 0.48422795 0.05811093
cat(sprintf("\nGQT speedup over GHK: %.1fx\n",
            mean(rt$runtime_sec[rt$method == "GHK"]) /
            mean(rt$runtime_sec[rt$method == "GQT"])))

##
## GQT speedup over GHK: 8.3x

# Max MAE difference
max_diff_pct <- max(abs(mae_wide$pct_diff), na.rm = TRUE)
cat(sprintf("\nMax |MAE difference| between GHK and GQT: %.2f%%\n", max_diff_pct))

##
## Max |MAE difference| between GHK and GQT: 33.44%

```

Likely findings to report

Fill in as the numbers come out:

1. **Convergence.** Both methods converge at ____% across the 18 cells.
2. **Runtime.** GQT is ~____x faster than GHK on average.
3. **Accuracy.** Across all 18 cells, GHK and GQT produce MAE values within ____% of each other for both parameters.
4. **Sample size effect.** MAE decreases by roughly a factor of ____ going from n=25 to n=49, consistent with a $\sqrt{n}$ convergence rate.
5. **Range effect.** MAE of the range parameter grows with the true range value — larger ranges are harder to identify at fixed n.
6. **Lambda effect.** Lower-intensity cells ($\lambda=1$) have [more/less] variability in estimates.
7. **Any surprises.** [Fill in any cell where GQT materially underperformed GHK, if any.]

Section 7: Tables for the report

LaTeX-ready tables. These can be pasted into the report directly (wrapped in a `\begin{tabular}` environment).

```

cat("\n% ---- Runtime table ----\n")

##
## % ---- Runtime table ----

rt_summary_simple <- aggregate(
  runtime_sec ~ n_total + method,
  data = rt,
  FUN = mean
)

for (i in seq_len(nrow(rt_summary_simple))) {

```

```

cat(sprintf("%d & %s & %.3f \\\n",
            rt_summary_simple$n_total[i],
            rt_summary_simple$method[i],
            rt_summary_simple$runtime_sec[i]))
}

```

```

## 25 & GHK & 0.270 \\  

## 49 & GHK & 0.698 \\  

## 25 & GQT & 0.032 \\  

## 49 & GQT & 0.085 \\  


```

```

cat("\n% ---- MAE table (Intercept) ----\n")

```

```

##  

## % ---- MAE table (Intercept) ----  

for (i in seq_len(nrow(mae_intercept))) {  

  cat(sprintf("%d & %g & %g & %s & %.4f \\\n",  
              mae_intercept$n_total[i],  
              mae_intercept$range[i],  
              mae_intercept$lambda[i],  
              mae_intercept$method[i],  
              mae_intercept$MAE[i]))  

}

```

```

## 25 & 0.1 & 1 & GHK & 0.1943 \\  

## 49 & 0.1 & 1 & GHK & 0.1732 \\  

## 25 & 0.3 & 1 & GHK & 0.3207 \\  

## 49 & 0.3 & 1 & GHK & 0.3103 \\  

## 25 & 0.5 & 1 & GHK & 0.5196 \\  

## 49 & 0.5 & 1 & GHK & 0.4129 \\  

## 25 & 0.1 & 5 & GHK & 0.0860 \\  

## 49 & 0.1 & 5 & GHK & 0.0724 \\  

## 25 & 0.3 & 5 & GHK & 0.1631 \\  

## 49 & 0.3 & 5 & GHK & 0.1537 \\  

## 25 & 0.5 & 5 & GHK & 0.2039 \\  

## 49 & 0.5 & 5 & GHK & 0.2034 \\  

## 25 & 0.1 & 10 & GHK & 0.0582 \\  

## 49 & 0.1 & 10 & GHK & 0.0573 \\  

## 25 & 0.3 & 10 & GHK & 0.1132 \\  

## 49 & 0.3 & 10 & GHK & 0.1016 \\  

## 25 & 0.5 & 10 & GHK & 0.1345 \\  

## 49 & 0.5 & 10 & GHK & 0.1396 \\  

## 25 & 0.1 & 1 & GQT & 0.1896 \\  

## 49 & 0.1 & 1 & GQT & 0.1671 \\  

## 25 & 0.3 & 1 & GQT & 0.2889 \\  

## 49 & 0.3 & 1 & GQT & 0.2693 \\  

## 25 & 0.5 & 1 & GQT & 0.3458 \\  

## 49 & 0.5 & 1 & GQT & 0.3550 \\  

## 25 & 0.1 & 5 & GQT & 0.0854 \\  

## 49 & 0.1 & 5 & GQT & 0.0730 \\  

## 25 & 0.3 & 5 & GQT & 0.1645 \\  

## 49 & 0.3 & 5 & GQT & 0.1550 \\  

## 25 & 0.5 & 5 & GQT & 0.2053 \\  

## 49 & 0.5 & 5 & GQT & 0.2006 \\  


```



```
## 25 & 0.1 & 10 & GQT & 0.0579 \\  
## 49 & 0.1 & 10 & GQT & 0.0574 \\  
## 25 & 0.3 & 10 & GQT & 0.1133 \\  
## 49 & 0.3 & 10 & GQT & 0.1027 \\  
## 25 & 0.5 & 10 & GQT & 0.1337 \\  
## 49 & 0.5 & 10 & GQT & 0.1399 \\  

```

End of K13A-GRID analysis

Next steps: - `k13a_pilot_analysis.Rmd` — Day-1 pilot (`sim_n=200`) as a single reference point, plus H18-B (`sim_n=1`) just for the figure. - `misspec_analysis.Rmd` — 2x2 factorial analysis of the MISSPEC study. - Then the missingness and zero-inflation extensions.